Conditional Computation Graphs and Distributed Optimizer Synchronization

Why per-leaf gradient allreduce fails, and the whole-model fix

Flux.jl Distributed Data Parallel Thesis Project

Technical report · Date: 2026-09-14

Audience: thesis author, before implementing M1.1

Scope. This report explains why Optimisers.jl and the current Flux `DistributedOptimizer` break when different ranks activate different conditional branches, and what the (currently dropped) whole-model synchronization solution looks like. It records the verified failure detector and the M1.1 design as of 2026-09-14. It does not claim any upstream status: the M1.1 spike is **not** part of the open PR 2694

Optimisers.jl 0.4.7 (`~/.julia/packages/Optimisers/ZOK0f`) ·
Dirty master `b633cdc7` · M1.1 spike `3370b910` (read-only reference)

Contents

1. Executive Summary 3
2. The Optimisers.jl Update Contract 4
   - 2.1. Setup produces a tree of `Leaf` nodes 4
   - 2.2. `update!` is two passes 4
   - 2.3. The skip semantics 5
3. The Current Distributed Integration 6
   - 3.1. `DistributedOptimizer` is a per-leaf rule 6
   - 3.2. Why this couples communication to the local graph 6
4. Failure Scenarios 7
5. Sequence Diagrams 8
   - 5.1. (a) Deadlock: rank 0 blocks on collective 3 8
   - 5.2. (b) Silent corruption: equal counts, wrong pairing 9
   - 5.3. (c) Presence-protocol decision flow (whole-model design) 10
6. Why `nothing -> zeros` Is Not Enough 11
7. The Whole-Model Synchronization Design (M1.1) 12
   - 7.1. Shape of the API 12
   - 7.2. Specialized `update`/`update!` 12
   - 7.3. The synchronization algorithm 12
   - 7.4. Properties of the design 13
8. Compatibility Surface 14
9. Evidence 16
   - 9.1. Detector: conditional-graph DDP failure modes 16
   - 9.2. Deadlock trace 16
   - 9.3. Corruption trace 16
   - 9.4. Control trace (symmetric, correct) 17
   - 9.5. M1.1 correctness suite 17
10. Open Questions and Next Steps 18

1. Executive Summary

Distributed Data Parallel (DDP) training replicates a model on every rank and averages gradients across ranks before each optimizer step. When every rank runs the same computation on the same set of parameters, the current Flux `DistributedOptimizer` is correct. When ranks activate different **conditional branches** — for example, a model with two alternative heads where rank 0 uses head A and rank 1 uses head B — the implementation fails in two distinct ways.

The root cause is structural, not numerical. Optimisers.jl calls the per-parameter `apply!` callback only for leaves that have a **local** gradient. Flux's `DistributedOptimizer` performs its `allreduce!` **inside** that callback. The collective schedule therefore becomes a function of which local gradients happen to be present, and the ranks disagree about that schedule. Because collectives are ordered and matched by call order, a disagreement produces either a hang or a silent wrong answer:

1. **Deadlock**: different numbers of local gradients, so one rank blocks forever on a collective that its peer never issues.
2. **Silent corruption**: the same number of local gradients, but the k-th collective carries a different parameter on each rank; MPI pairs them and both ranks receive a wrong average with no error.

A detector (`scripts/checks/conditional_deadlock_check.jl`, run 2026-09-14) reproduces both failures and a correct symmetric control: 12 of 12 assertions pass in 2m27.0s. The detector deliberately treats a **hang** as a pass, because a hang is the documented behavior of the current code.

The obvious patch — replace `nothing` gradients with zero-filled arrays before the collective (`resolve_unused_parameters!!`) — repairs the deadlock but breaks correctness. A globally absent parameter would then present a zero gradient to Optimisers, so `_update!` would apply a step and advance the optimizer state (for Adam, the first and second moment estimates) even though no rank used the parameter. Correct DDP must preserve **skip** semantics for globally absent parameters.

The dropped whole-model design (M1.1, branch `ddp/integration-m1-spike` at `3370b910`) moves synchronization out of the per-leaf callback and into a specialized `update`/`update!` on a `DistributedOptimizerState` wrapper. It walks the whole model and gradient trees exactly as `Optimisers.setup` does, uses one presence `allreduce` per step to decide globally which parameters are present, allreduces each globally present parameter in model order, fills locally absent contributions with zeros, and emits `ZeroTangent` for globally absent parameters so the optimizer step is skipped and the state is untouched. Communication stays outside automatic differentiation, and only public Optimisers APIs are used.

This report gives the mental model, the failure taxonomy, the native sequence diagrams, the evidence, and the compatibility surface that any implementation must preserve.

2. The Optimisers.jl Update Contract

The failure only makes sense once the Optimisers.jl update contract is precise. The relevant code is Optimisers.jl 0.4.7 (the version pinned by this project's `Manifest.toml`; `~/.julia/packages/Optimisers/ZOK0f`). Line numbers below refer to that checkout.[1]

2.1. Setup produces a tree of `Leaf` nodes

`Optimisers.setup(rule, model)` (`interface.jl:29-58`) walks the model through `Optimisers.trainable`, stops at `Optimisers.isnumeric` leaves (arrays of non-integer numbers that Functors treats as leaves), and returns a tree of `Leaf(rule, state)` objects that mirrors the model structure. A cache keyed by identity makes tied, mutable parameters share one `Leaf`; isbits parameters are not cached and therefore get one `Leaf` per occurrence.

```
mutable struct Leaf{R,S}   # identity encodes parameter sharing
  rule::R
  state::S
  frozen::Bool
end

function setup(rule::AbstractRule, model)
  cache = IdDict()
  tree = _setup(rule, model; cache)
  isempty(cache) && @warn "setup found no trainable parameters in this model"
  tree
end
```

2.2. `update!` is two passes

`update`/`update!` (`interface.jl:70-101`) do not walk the model once. The first pass, `_grads!`, accumulates gradients into an `IdDict{Leaf,Any}`. The second pass, `_update!`, consumes that dictionary and applies each rule.

```
function update!(tree, model, grad, higher...)
  # First walk accumulates the gradient, keyed by Leaf identity:
  grads = IdDict{Leaf, Any}()
  _grads!(grads, tree, model, grad, higher...)
  # Second walk updates the model:
  newmodel = _update!(tree, model; grads, params = IdDict())
  tree, newmodel
end
```

The crucial methods are the ones that handle **absence**:

```
const Zero = Union{Nothing, AbstractZero}          # interface.jl:6

# If AD returns a Zero for a leaf, no dict entry is created:
_grads!(dict::IdDict, ℓ::Leaf, x, ::Zero...) = nothing   # interface.jl:108

# A Zero at a non-leaf node ends the recursion entirely:
_grads!(dict::IdDict, t, x, ::Zero...) = nothing         # interface.jl:119

subtract!(x, x̄::Zero) = x                                # interface.jl:104
```

The second pass checks for the dictionary entry:

[1] Optimisers 0.4.8 and 0.4.9 insert eight lines earlier in `interface.jl`, so the same methods appear at `subtract!` 112, the leaf `_grads!` at 116, and the non-leaf `_grads!` at 127. The 0.4.7 numbers are the ones that match this project's pinned `Manifest.toml`.

```
function _update!(ℓ::Leaf, x; grads, params)
  ℓ.frozen && return x
  params[(ℓ,x)] = if haskey(grads, ℓ)
    ℓ.state, x̄′ = apply!(ℓ.rule, ℓ.state, x, grads[ℓ]...)
    subtract!(x, x̄′)
  else
    x            # no gradient seen: leave parameter and state untouched
  end
end
```

2.3. The skip semantics

The contract can be stated in one sentence: **if a leaf's local gradient is `nothing` or any other `AbstractZero`, then `_grads!` records no entry, `_update!` does not call `apply!` for that leaf, and the optimizer state attached to that `Leaf` is left exactly as it was.** This is intentional, not an accident; it is the documented behavior for unused parameters (Optimisers.jl issue 140, referenced in the comments at `interface.jl:106-107` and `interface.jl:116-118`).

This is the property that a correct distributed integration must preserve **globally**. Locally absent must become globally absent when no rank used the parameter; only then is the step legitimately skipped on every rank.

3. The Current Distributed Integration

The current Flux implementation lives in `ddp_flux/src/distributed/public_api.jl` on dirty master `b633cdc7`. The pushed PR 2694 salvage branch contains the same per-leaf `allreduce!` without the `no_sync` experiment flag.

3.1. `DistributedOptimizer` is a per-leaf rule

`DistributedOptimizer` is a subtype of `Optimisers.AbstractRule`. Its `init` delegates to the wrapped optimizer, and its `apply!` — the per-leaf callback — performs the `allreduce!`:

```
struct DistributedOptimizer{B <: AbstractFluxDistributedBackend} <: AbstractRule
    backend::B
    opt
    no_sync::Bool
end

function Optimisers.apply!(opt::DistributedOptimizer, state, x, y)
    if opt.no_sync
        return Optimisers.apply!(opt.opt, state, x, y)
    else
        y_avg = DistributedUtils.allreduce!(opt.backend, y, DistributedUtils.avg)
        return Optimisers.apply!(opt.opt, state, x, y_avg)
    end
end

Optimisers.init(opt::DistributedOptimizer, x::AbstractArray) =
    Optimisers.init(opt.opt, x)
```

The `no_sync` flag is a dirty-master experiment to disable synchronization for local debugging; it is not a correctness mechanism. The salvage branch has the same `allreduce!` call without the flag.

3.2. Why this couples communication to the local graph

Because `apply!` is invoked by `_update!` only for leaves present in the `grads` dictionary, the `allreduce!` calls happen **only when Optimisers decided a local gradient exists**. The collective schedule is therefore exactly the sequence of locally present trainable leaves, in model order. Two consequences follow:

1. If every rank has the same set of locally present leaves, the schedules match and the result is correct.
2. If the ranks differ, the schedules differ. Ordered collectives (`MPI_Allreduce`, NCCL) match by call order, not by parameter identity, so a mismatch is either a hang or a wrong pairing.

Two other helpers exist in the same file but are **not** part of the automatic path: `sync_gradients!` (a manual `fmap` over an explicit gradient tree) and `resolve_unused_parameters!!` (the `nothing -> zeros` patch discussed in Section 6). Neither repairs the root cause on its own.

4. Failure Scenarios

The table below enumerates the behaviors. The "collective behavior" column is what actually happens at the MPI/NCCL level; the "outcome" column is what the user observes.

Scenario	Rank behavior	Collective behavior	Outcome
Uniform	Every rank activates the same full set of parameters.	Identical leaf sequence on every rank; the k-th collective is the same parameter everywhere.	Allreduces pair correctly; averaged gradients are correct. No error.
Silent corruption	Rank 0 uses head A; rank 1 uses head B. The two branches have the same number of trainable leaves.	Equal collective counts, but the k-th collective carries a different parameter on each rank.	MPI pairs rank 0's A-gradient with rank 1's B-gradient. Both ranks receive the wrong average and continue training. **No error is raised.**
Deadlock	Rank 0 uses trunk + A + B (3 leaves); rank 1 uses trunk + C (2 leaves).	Rank 0 issues 3 `allreduce!` calls; rank 1 issues 2.	Rank 0 blocks forever on its 3rd collective, which rank 1 never issues. Rank 1 finishes and waits at the next barrier. The job hangs.
Globally absent	Both ranks skip the same parameter (the branch is inactive everywhere).	Neither rank issues a collective for that parameter.	Consistent. `_update!` finds no dictionary entry, so it skips the leaf and leaves the optimizer state untouched. **Correct.**
Non-trainable / tied / wrapped / frozen	Ranks may differ in which leaves exist, how tied gradients combine, or which wrappers are present: BatchNorm μ/σ², tied weights, `Transpose`/`Adjoint`, frozen leaves.	Per-leaf presence differs, or the same parameter is reduced a different number of times, or under a different wrapper.	Mismatch: deadlock or silent corruption. Any implementation must mirror `Optimisers.setup` traversal (BatchNorm μ/σ² excluded), deduplicate tied parameters by identity, and pair wrappers in parent space.

Table 1: Failure taxonomy of the current per-leaf `DistributedOptimizer`. The two dangerous rows are silent corruption (wrong answer, no error) and deadlock (hang). Globally absent is the one case the current code handles correctly, and any fix must not regress it.

The corruption and deadlock rows are not hypothetical. Both are reproduced by the detector in Section 9.

5. Sequence Diagrams

All diagrams use only native Typst primitives (`box`, `rect`, `line`, `polygon`, `place`, `text`). In the timeline diagrams, time increases downward. A **double-headed** arrow is a collective issued by both ranks and matched by MPI; a **single-headed** arrow is a collective issued by only one rank, which therefore has no partner.

5.1. (a) Deadlock: rank 0 blocks on collective 3

In `deadlock` mode the probe gives rank 0 the loss over `trunk`, `a`, `b` and rank 1 the loss over `trunk`, `c`. Optimisers calls `apply!` in model order for locally present leaves only: rank 0 issues collectives for `trunk`, `a`, `b`; rank 1 issues collectives for `trunk`, `c`. The first two pair (and the second pair is already mismatched, `a` against `c`); the third on rank 0 has no partner, so rank 0 blocks forever while rank 1 completes and waits at the barrier.

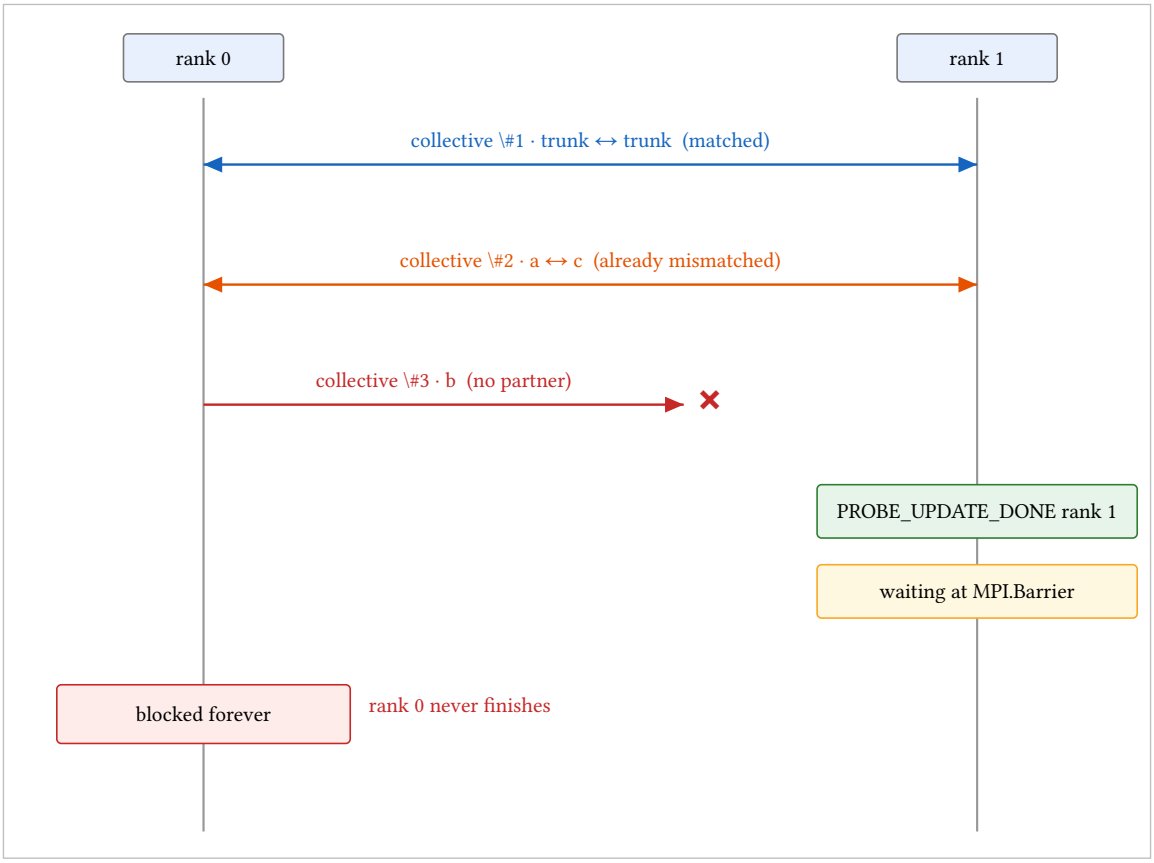


Figure 1: **(a) Deadlock.** Asymmetric collective counts. Rank 0's third collective never matches; the job hangs. MPI has no notion of a parameter name, so it cannot detect the mismatch.

5.2. (b) Silent corruption: equal counts, wrong pairing

In `corruption` mode rank 0 computes the loss over `trunk`, `a` and rank 1 over `trunk`, `b`. Both ranks issue exactly two collectives, so nothing hangs. But the second collective carries `a` on rank 0 and `b` on rank 1. MPI averages the two buffers elementwise and hands the same wrong vector to both ranks, which then apply it to **different** parameters.

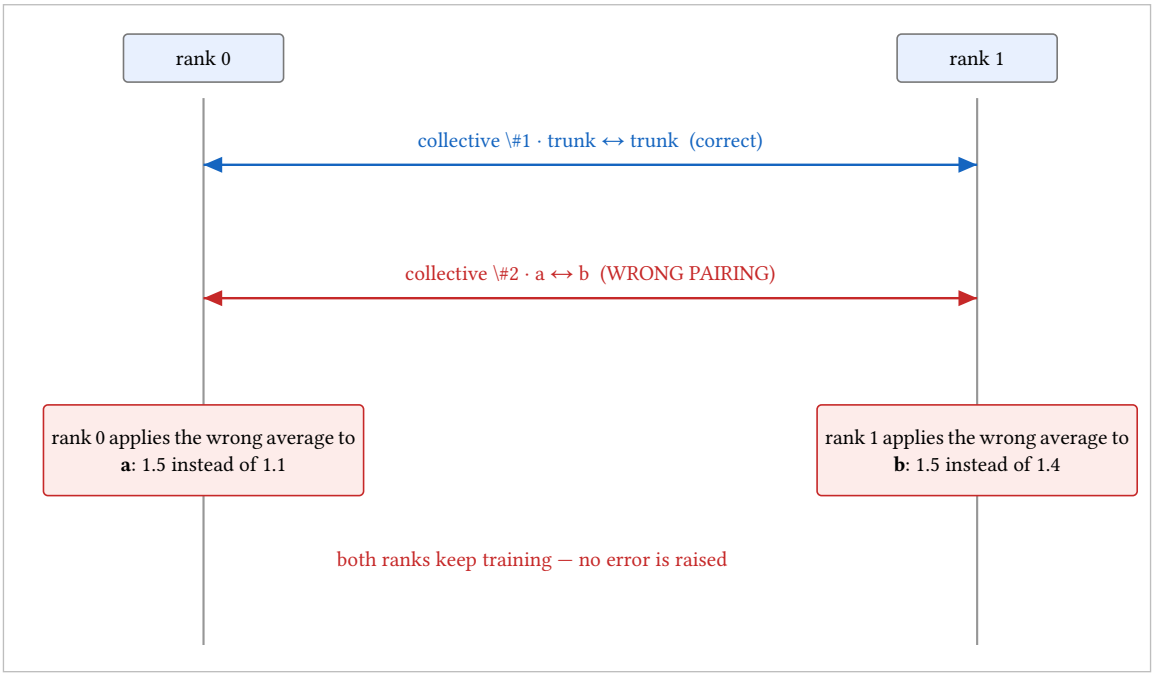
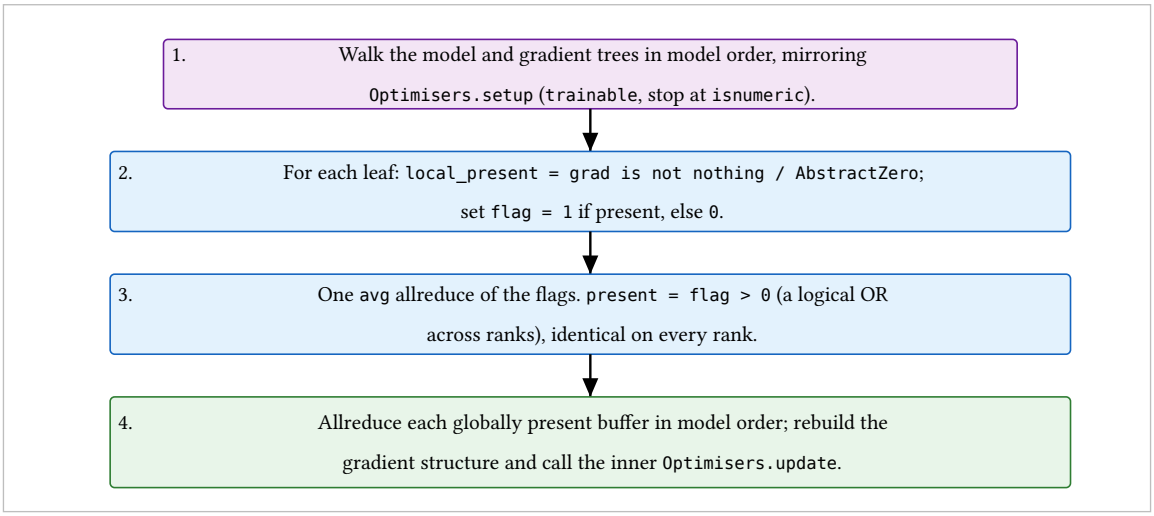


Figure 2: **(b) Silent corruption.** Equal collective counts. MPI matches by call order, so the collective for `a` pairs with the collective for `b`. Both parameters receive the wrong averaged gradient and training continues.

5.3. (c) Presence-protocol decision flow (whole-model design)

The whole-model design replaces the per-leaf schedule with a single, globally agreed schedule. The flow below is the decision logic for one trainable leaf. The key point is the last row: when a leaf is absent on **every** rank, the design returns `ZeroTangent`, so Optimisers skips the step and leaves the optimizer state untouched.



	locally present	**locally absent**
globally present	Allreduce the real gradient; apply the optimizer step.	Contribute zeros to the allreduce; apply the optimizer step.
globally absent	Cannot occur: the presence flag would be 1.	Return `ZeroTangent`. `_update!` skips the step; Adam state is untouched.

Listing 1: **(c) Presence-protocol decision flow.** One presence allreduce per step makes the schedule globally consistent. Locally absent but globally present contributes zeros (no hang). Globally absent returns `ZeroTangent` (no wrong step). The upper-right cell is unreachable by construction.

6. Why `nothing -> zeros` Is Not Enough

The intuitive patch is to replace every `nothing` gradient with a zero-filled array before the collective. This is exactly what `resolve_unused_parameters!!(backend, gs, model)` did in the earlier design:

```
function resolve_unused_parameters!!(backend, gs, model)
    function walk(g, p)
        if p isa AbstractArray
            return g === nothing ? fill!(similar(p), 0) : g
        end
        # ... recurse, filling absent subtrees with zeros ...
    end
    return walk(gs, model)
end
```

This does repair the deadlock: after zero-filling, every rank presents a gradient for every trainable leaf, so every rank issues the same collectives in the same order. But it destroys the **skip** semantics that the Optimisers contract depends on.

Once a zero-filled array reaches `_grads!`, it is a real gradient, not a `Zero`. An entry is created in the `grads` dictionary, `_update!` calls `apply!`, and the wrapped optimizer advances. For Adam this is not a no-op:

Adam's state is `(mt, vt, βt)`, where `βt` is the running `β^t` used for bias correction. Calling `apply!` with a zero gradient still:

1. decays `mt` and `vt` (`mt = β[1] * mt`, `vt = β[2] * vt`),
2. advances `βt .= βt .* β`, which changes the bias correction, and
3. if the parameter ever had a nonzero gradient (`mt ≠ 0`), returns a nonzero step $-\eta \frac{m_t}{\sqrt{v_t}+\varepsilon}$, so the parameter drifts.

The damage accumulates. A parameter that no rank has used for many steps — for example the inactive head of a conditional model — drifts because its Adam state keeps changing, and the drift is different from the single-process reference. The failure is quiet and only visible when comparing optimizer states, not losses.

There are three further problems with the zero-fill approach:

1. **It allocates on every step.** Fresh zero buffers are allocated for absent leaves, including on GPU devices where allocation is expensive.
2. **It cannot express "absent everywhere".** Zero-filling is unconditional; there is no mechanism to distinguish "absent here but used elsewhere" (which should contribute zero) from "absent everywhere" (which should skip).
3. **It does not address tied or wrapped parameters.** A zero-fill walk over the gradient structure can duplicate or miss contributions for tied parameters and can misplace gradients for `Transpose`/`Adjoint` wrappers.

The correct rule is therefore the one in the Optimisers contract, lifted to the global level: **skip when absent everywhere; contribute zero when present somewhere.** A naive zero-fill collapses both cases into "present", which is precisely the case that must not be collapsed.

7. The Whole-Model Synchronization Design (M1.1)

The design below is implemented on branch `ddp/integration-m1-spike` at commit `3370b910` in `/home/kurapica/Projects/ddp_flux/flux-integration` (read-only reference for this report). It is **not** part of PR 2694 the salvage PR dropped unused-parameter support to keep its scope clean. The design is recorded here because it is the reference solution for conditional graphs.

7.1. Shape of the API

`DistributedOptimizer` remains a subtype of `Optimisers.AbstractRule`, so it is still constructed the same way. What changes is that `setup` on a `DistributedOptimizer` returns a dedicated state wrapper instead of a tree of per-leaf rules:

```
struct DistributedOptimizer{B <: AbstractFluxDistributedBackend} <: AbstractRule
    backend::B
    opt
end

struct DistributedOptimizerState{B <: AbstractFluxDistributedBackend, T}
    backend::B
    tree::T
end

@functor DistributedOptimizerState (tree,)

Optimisers.setup(opt::DistributedOptimizer, model) =
    DistributedOptimizerState(opt.backend, Optimisers.setup(opt.opt, model))
```

The inner optimizer state is set up by the normal Optimisers path (`opt.opt`), so per-leaf `Leafs`, tied-parameter sharing, and freezing all behave exactly as in single-process training.

7.2. Specialized `update`/`update!`

Synchronization happens once, on the whole model, before the inner update runs:

```
function Optimisers.update!(s::DistributedOptimizerState, model, grads, higher...)
    isempty(higher) || throw(ArgumentError(_HIGHER_ORDER_ERROR))
    grads2 = _sync_gradients(s.backend, model, grads)
    _, model2 = Optimisers.update!(s.tree, model, grads2)
    return (s, model2)
end

function Optimisers.update(s::DistributedOptimizerState, model, grads, higher...)
    isempty(higher) || throw(ArgumentError(_HIGHER_ORDER_ERROR))
    grads2 = _sync_gradients(s.backend, model, grads)
    tree2, model2 = Optimisers.update(s.tree, model, grads2)
    return (DistributedOptimizerState(s.backend, tree2), model2)
end
```

The important structural fact is that `_sync_gradients` produces a gradient tree with exactly the shape that `Optimisers.update!` already understands — including `nothing` and `AbstractZero` nodes — so the inner update is the unmodified, public Optimisers path. No private Optimisers internals are used.

7.3. The synchronization algorithm

`_sync_gradients` has three phases.

Phase 1 — walk and buffer. `_walk!` mirrors `Optimisers.setup`: it descends through `Optimisers.trainable`, stops at `Optimisers.isnumeric` leaves, and allocates one fresh writable buffer per unique mutable trainable parameter, in model order. Tied mutable parameters are deduplicated by

identity (`IdDict`); isbits parameters are not deduplicated and get one buffer per occurrence, matching Optimisers' `Leaf` caching. The contribution of each occurrence is copied into a fresh `similar(x)` temporary and accumulated, so caller gradients are never mutated.

Phase 2 — presence protocol. One `Float32` `avg` allreduce of $\frac{0}{1}$ flags decides, identically on every rank, which parameters are present **anywhere**:

```
np = length(bufs)
np == 0 && return grads          # no trainable parameters: no collectives

pres = zeros(Float32, np)
for i in 1:np
    pres[i] = bufs[i].present ? 1f0 : 0f0
end
DistributedUtils.allreduce!(backend, pres, DistributedUtils.avg)
present = pres .> 0              # threshold is > 0, not > 0.5

for i in 1:np
    present[i] && DistributedUtils.allreduce!(backend, bufs[i].buf,
DistributedUtils.avg)
end
```

The threshold `> 0` is important for more than two ranks: `avg` of $\frac{0}{1}$ flags is the fraction of ranks that saw the parameter, so any fraction strictly greater than zero means "present somewhere". `> 0.5` would be wrong for three or more ranks.

Every globally present buffer is allreduced in strictly ascending index, which is the model-structure order; every rank therefore calls every collective in the same order.

Phase 3 — rebuild. `_build!` reconstructs a gradient structure that mirrors the model's structural children. Non-trainable children and nodes without trainable descendants become `nothing` (as Zygote would produce). A trainable leaf gets the allreduced buffer when it is globally present, and `CRC.ZeroTangent()` otherwise:

```
return present[i] ? bufs[i].buf : CRC.ZeroTangent()
```

For a tied parameter, the buffer is emitted only at the first occurrence; later occurrences return `ZeroTangent`, so Optimisers' per-`Leaf` accumulation does not double-count.

7.4. Properties of the design

1. **Communication is outside AD.** The collectives run in `update`/`update!`, not inside a differentiated function. Gradients arrive from Zygote/Enzyme as ordinary arrays.
2. **Public APIs only.** The only Optimisers functions used are `setup`, `update`, `update!`, `init`, `apply!` (inside the inner rule), `trainable`, `isnumeric`, `adjust`, `freeze!`, and `thaw!`.
3. **One global schedule.** The collective count is `1` (presence) plus the number of globally present unique trainable parameters — identical on every rank by construction, regardless of which branches each rank activated.
4. **Skip semantics preserved.** Globally absent parameters never enter the inner update as gradients, so their state is untouched.
5. **Device-preserving.** Buffers use `similar(p)` and `copyto!`; there is no `collect` and no silent GPU-to-CPU transfer.

8. Compatibility Surface

Any implementation of whole-model synchronization must preserve the behaviors below. The table records the requirement and how M1.1 satisfies it.

Surface	Requirement	M1.1 handling
Trainable semantics	Only leaves that `Optimisers.setup` would optimize may be synchronized. BatchNorm running statistics μ/σ² are not trainable.	Walk via `Optimisers.trainable`, stop at `Optimisers.isnumeric`; proven by a counting-backend test that BatchNorm μ/σ² are untouched.
Immutable model values	`update!` must return the model produced by the inner update, not discard it.	`_, model2 = Optimisers.update!(s.tree, model, grads2); return (s, model2);` `update` rebuilds the state wrapper with `tree2`.
Transpose / Adjoint	Gradients must be paired with the model node in parent space, not through the wrapper.	Structural pairing with `functor(typeof(model_node), base(grad_node))[1]`, the same alignment Optimisers uses; matrix and row-vector cases tested analytically.
Tied parameters	A shared mutable parameter must be reduced once, not once per occurrence.	Deduplicated by identity (`IdDict`); the allreduced buffer appears at the first occurrence, later ones return `ZeroTangent` so `_grads!` does not double-count.
isbits parameters	No identity to tie on; one `Leaf` per occurrence in Optimisers.	One buffer per occurrence, `Array(x)` instead of `similar`; a `FillArrays.Fill` test confirms no `objectid` collision.
Frozen leaves	`freeze!` must prevent an update for that leaf.	`freeze!`/`thaw!` delegate to the inner tree; the inner `_update!` handles frozen leaves as usual.
Device-preserving buffers	No silent GPU-to-CPU transfer; buffers must match the parameter's device and element type.	`similar(p)` plus `copyto!`; `collect(g)` was removed.
Composition	Nesting `DistributedOptimizer` inside `OptimiserChain` (or vice versa) must not silently disable synchronization.	`Optimisers.init` and `apply!` on `DistributedOptimizer` throw `ArgumentError`; the rule must be outermost. Top-level setup goes through the specialized method.
Higher-order grads	Out of scope for the first-order baseline.	`isempty(higher) || throw(ArgumentError(...))` in both `update` methods.
Enzyme `Duplicated`	Not supported for update; setup must not silently mis-handle it.	Setup unwraps `model.val`; `update`/`update!` on a `Duplicated` model throw `ArgumentError`.

Table 2: Compatibility surface of the whole-model synchronization design. Each row is a property that the single-process Optimisers path relies on and that a distributed implementation must not break.

9. Evidence

9.1. Detector: conditional-graph DDP failure modes

The detector consists of two files created for this investigation:

1. `scripts/checks/conditional_deadlock_probe.jl` — the child, run under `mpiexec`. It builds a four-vector model (`trunk`, `a`, `b`, `c`), selects a branch set by rank and mode, wraps the backend in a logging decorator that prints every collective before it happens, and prints machine-readable markers.
2. `scripts/checks/conditional_deadlock_check.jl` — the driver, a single process. It launches the child, waits for both ranks to report `PROBE_READY`, then uses a timer to declare a hang if the update does not finish. A hang is a **pass**: it is the documented behavior of the current implementation.

Exact command (run 2026-09-14):

```
cd /home/kurapica/Projects/ddp_flux/ddp-flux-thesis
julia --project=. scripts/checks/conditional_deadlock_check.jl
```

Result:

```
Test Summary: conditional-graph DDP failure modes | 12 pass, 12 total, 2m27.0s
DETECTOR SUMMARY: current implementation confirmed broken (as documented)
```

The driver exits 0 when the current implementation behaves as documented. When a fix lands, `deadlock` mode will complete instead of hanging, and the detector must be updated.

9.2. Deadlock trace

```
PROBE_READY rank=0
PROBE_READY rank=1
TRACE rank=0 allreduce #1 size=4
TRACE rank=1 allreduce #1 size=4
TRACE rank=0 allreduce #2 size=4
TRACE rank=1 allreduce #2 size=4
TRACE rank=0 allreduce #3 size=4
PROBE_UPDATE_DONE rank=1
[driver kills the job: rank 0 never finishes]
```

Rank 0 issued a third collective; rank 1 completed after two. Rank 0 is blocked on the unmatched third collective. The detector asserts exactly this shape: `count_allreduces(out, 0) == 3`, `count_allreduces(out, 1) == 2`, rank 1 done, rank 0 never done.

9.3. Corruption trace

```
TRACE rank=0 allreduce #1 size=4
TRACE rank=1 allreduce #1 size=4
TRACE rank=0 allreduce #2 size=4
TRACE rank=1 allreduce #2 size=4
PROBE_UPDATE_DONE rank=0
PROBE_UPDATE_DONE rank=1
PROBE_VERDICT mode=corruption a_expected=1.1 a_actual=1.5 b_expected=1.4 b_actual=1.5
corrupted=true
```

Both ranks completed with equal collective counts, so no hang. The second collective paired rank 0's `a`-gradient with rank 1's `b`-gradient. The expected updates (if each parameter had been averaged correctly against a zero contribution from the rank that did not use it) are 1.1 for `a` and 1.4 for `b`; both actual values are 1.5. `corrupted=true`.

9.4. Control trace (symmetric, correct)

```
TRACE rank=0 allreduce #1..#4 size=4
TRACE rank=1 allreduce #1..#4 size=4
PROBE_VERDICT mode=control correct=true a=1.2 b=1.8 c=1.4
```

With every rank using every parameter, the four collectives pair correctly and the update matches the analytic expectation. This is the baseline the detector uses to show that the failures are caused by the conditional graph and not by the harness.

9.5. M1.1 correctness suite

The M1.1 spike was validated with the direct ext_distributed runner on CPU/MPI (the evidence recorded in `artifacts/logs/m1.1/`):

```
cd /home/kurapica/Projects/ddp_flux/flux-integration
JULIA_MPI_TEST_NPROCS=2 julia --project=test test/ext_distributed/runtests.jl mpi
JULIA_MPI_TEST_NPROCS=4 julia --project=test test/ext_distributed/runtests.jl mpi
```

Both runs reported 6 of 6 files passing (1m35.6s at two ranks, 1m59.7s at four ranks). The new `conditional_distributedtest.jl` covers real-AD branches with an analytic global-mean reference, Adam-versus-reference moment states, globally absent skip semantics, BatchNorm collective counts, transpose/adjoint pairing, tied and isbits parameters, composition rejection, and gradient non-mutation.

These results are for the M1.1 spike, not for the open PR. The current PR 2694 salvage branch does not contain unused-parameter support, and the detector above therefore still reports the broken behavior on it.

10. Open Questions and Next Steps

1. **Fold M1.1 into the PR or keep it as a spike?** M1.1 is unpushed and not part of PR 2694 The salvage PR deliberately dropped unused-parameter support. The next decision is whether to extend the spike, fold it into the PR as a separate commit, or keep it as a documented reference.
2. **NCCL runtime validation.** The M1.1 design is API-compatible with NCCL and the presence protocol uses an `avg` allreduce that works on every backend, but the NCCL path has only been exercised on CPU/MPI. It must be validated on GPUs before any performance claim.
3. **Performance.** `_sync_gradients` copies every gradient into a fresh `similar(p)` buffer. In-place reuse is a deferred optimization, to be done only after correctness is settled. The extra presence allreduce per step is one small `Float32` buffer and is expected to be negligible.
4. **Frozen leaves under the presence protocol.** The interaction between `freeze!`, the presence flag, and the inner update should be pinned by a test that freezes a subset of leaves on all ranks.
5. **The `no_sync` experiment.** The dirty-master `no_sync` flag is a debugging convenience with no correctness role. Decide whether it survives into any merged design.
6. **Higher-order gradients and Enzyme.** Both are explicitly rejected by M1.1. If the thesis needs them, the design must be extended deliberately rather than silently dropping synchronization.
7. **Composition semantics.** M1.1 rejects nesting. A future design could support `OptimiserChain` by synchronizing the whole model before the chain runs, but that requires an explicit decision about which chain member owns synchronization.
8. **Documentation.** `docs/src/guide/gpu.md:366-377` still shows pre-M1 `Leaf` output and is deferred.
9. **Legacy API.** `resolve_unused_parameters!!` is deleted on the M1.1 branch because the automatic path supersedes it; any merged design should remove it rather than leave two competing mechanisms.

Technical report generated from the `ddp-flux-thesis` workspace.

Detector run: 2026-09-14 · Optimisers.jl 0.4.7 · M1.1 spike `3370b910`

M1.1 is a read-only reference and is not part of PR 2694